PIZZA STATION ANALYSIS

NAME -: ARPITA PRAVIN BINJWE.

MODULE NAME -: SQL

Project Description

The **Pizza Station Project** is a data analysis project created using **SQL** to understand business performance and customer ordering patterns of a pizza restaurant. The main goal of this project is to analyze sales data and generate meaningful insights that can help improve decision-making and increase revenue.

In this project, different SQL queries are used to retrieve important information such as total number of orders, total revenue, most popular pizza types, common pizza sizes, and category-wise performance. The project also includes intermediate and advanced level queries like calculating percentage contribution of each pizza type, analyzing cumulative revenue over time, and identifying top performing pizzas based on revenue.

By working on this dataset, we learn how to use SQL concepts like **JOIN, GROUP BY, ORDER BY, Aggregate Functions, Subqueries, and Window Functions** to solve real-world business problems.

This project helps in understanding:

- Customer preferences
- Best selling pizza types
- Sales trends over time
- Revenue contribution by different categories
- Business growth patterns

Overall, this project demonstrates how **data analysis using SQL** can provide valuable insights for business improvement and strategic planning.

Basic:

1. Retrieve the total number of orders placed.

```
SELECT COUNT(ORDER_ID) AS TOTAL_ORDERS
FROM ORDERS;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

TOTAL_ORDERS
21350

---

2. Calculate the total revenue generated from pizza sales.

```
SELECT SUM(od.quantity * p.price) AS TOTAL_REVENUE
FROM ORDER_DETAILS OD
JOIN PIZZAS P
ON od.pizza_id = p.pizza_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

TOTAL_REVENUE
817860.049999993

---

3. Identify the highest-priced pizza.

```
SELECT PIZZA_ID, PRICE
FROM PIZZAS
ORDER BY PRICE DESC
LIMIT 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

PIZZA_ID	PRICE
the_greek_xxl	35.95

4. Identify the most common pizza size ordered.

```
SELECT P.SIZE, COUNT(*) AS TOTAL_ORDERS
FROM ORDER_DETAILS OD
JOIN PIZZAS P
ON OD.PIZZA_ID = P.PIZZA_ID
GROUP BY P.SIZE
ORDER BY TOTAL_ORDERS DESC
LIMIT 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

size	total_orders
L	18526

---

5. List the top 5 most ordered pizza types along with their quantities.

```
SELECT pt.name, SUM(od.quantity) AS total_quantity
FROM order_details od
JOIN pizzas p
ON od.pizza_id = p.pizza_id
JOIN pizza_types pt
ON p.pizza_type_id = pt.pizza_type_id
GROUP BY pt.name
ORDER BY total_quantity DESC
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	total_quantity
The Classic Deluxe Pizza	2453
The Barbecue Chicken Pizza	2432
The Hawaiian Pizza	2422
The Pepperoni Pizza	2418
The Thai Chicken Pizza	2371

Intermediate:

6. Join the necessary tables to find the total quantity of each pizza category ordered.

```
SELECT pt.category, SUM(od.quantity) AS total_quantity
FROM order_details od
JOIN pizzas p
ON od.pizza_id = p.pizza_id
JOIN pizza_types pt
ON p.pizza_type_id = pt.pizza_type_id
GROUP BY pt.category
ORDER BY total_quantity DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	total_quantity
Classic	14888
Supreme	11987
Veggie	11649
Chicken	11050

---

7. Determine the distribution of orders by hour of the day.

```
SELECT HOUR(time) AS order_hour,
COUNT(order_id) AS total_orders
FROM orders
GROUP BY order_hour
ORDER BY order_hour;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_hour	total_orders
9	1
10	8
11	1231
12	2520
13	2455
14	1472
15	1468

---

8. Join relevant tables to find the category-wise distribution of pizzas.

```
SELECT pt.category,
COUNT(DISTINCT od.order_id) AS total_orders
FROM order_details od
JOIN pizzas p
ON od.pizza_id = p.pizza_id
JOIN pizza_types pt
ON p.pizza_type_id = pt.pizza_type_id
GROUP BY pt.category
ORDER BY total_orders DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	total_orders
Classic	10859
Supreme	9085
Veggie	8941
Chicken	8536

---

9. Group the orders by date and

```
SELECT
AVG(daily_total) AS avg_pizzas_per_day
FROM (
SELECT
o.order_date,
SUM(od.quantity) AS daily_total
FROM orders o
JOIN order_details od
ON o.order_id = od.order_id
GROUP BY o.order_date
)AS daily_orders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	avg_pizzas_per_day
▶	138.4749

---

10. Determine the top 3 most ordered pizza types based on revenue.

```
SELECT
pt.name AS pizza_name,
SUM(od.quantity * p.price) AS total_revenue
FROM order_details od
JOIN pizzas p
ON od.pizza_id = p.pizza_id
JOIN pizza_types pt
ON p.pizza_type_id = pt.pizza_type_id
GROUP BY pt.name
ORDER BY total_revenue DESC
LIMIT 3;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	pizza_name	total_revenue
▶	The Thai Chicken Pizza	43434.25
	The Barbecue Chicken Pizza	42768
	The California Chicken Pizza	41409.5

Advanced:

11. Calculate the percentage contribution of each pizza type to total revenue.

```
SELECT
pt.name,
SUM(od.quantity * p.price) AS revenue,
ROUND(
(SUM(od.quantity * p.price) /
```

```
(SELECT SUM(od2.quantity * p2.price)
FROM order_details od2
JOIN pizzas p2 ON od2.pizza_id = p2.pizza_id)
) * 100, 2
) AS revenue_percentage
FROM order_details od
JOIN pizzas p ON od.pizza_id = p.pizza_id
JOIN pizza_types pt ON p.pizza_type_id = pt.pizza_type_id
GROUP BY pt.name
ORDER BY revenue_percentage DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	revenue	revenue_percentage
The Thai Chicken Pizza	43434.25	5.31
The Barbecue Chicken Pizza	42768	5.23
The California Chicken Pizza	41409.5	5.06
The Classic Deluxe Pizza	38180.5	4.67
The Spicy Italian Pizza	34831.25	4.26
The Southwest Chicken Pizza	34705.75	4.24
The Italian Supreme Pizza	33476.75	4.09

---

12. Analyze the cumulative revenue generated over time.

```
SELECT
o.date,
SUM(od.quantity * p.price) AS daily_revenue,
SUM(SUM(od.quantity * p.price)) OVER (
ORDER BY o.date
) AS cumulative_revenue
FROM orders o
JOIN order_details od ON o.order_id = od.order_id
JOIN pizzas p ON od.pizza_id = p.pizza_id
GROUP BY o.date
```

ORDER BY o.date;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

date	daily_revenue	cumulative_revenue
2015-01-01	2713.8500000000004	2713.8500000000004
2015-01-02	2731.8999999999996	5445.75
2015-01-03	2662.4	8108.15
2015-01-04	1755.4500000000003	9863.6
2015-01-05	2065.95	11929.55
2015-01-06	2428.95	14358.5
2015-01-07	2202.2000000000003	16560.7

---

13. Determine the top 3 most ordered pizza types based on revenue for each pizza category.

```
SELECT category, name, revenue
FROM (
SELECT
pt.category,
pt.name,
SUM(od.quantity * p.price) AS revenue,
RANK() OVER (
PARTITION BY pt.category
ORDER BY SUM(od.quantity * p.price) DESC
) AS rank_position
FROM order_details od
JOIN pizzas p ON od.pizza_id = p.pizza_id
JOIN pizza_types pt ON p.pizza_type_id = pt.pizza_type_id
GROUP BY pt.category, pt.name
) ranked_data
WHERE rank_position <= 3;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	name	revenue
Chicken	The Thai Chicken Pizza	43434.25
Chicken	The Barbecue Chicken Pizza	42768
Chicken	The California Chicken Pizza	41409.5
Classic	The Classic Deluxe Pizza	38180.5
Classic	The Hawaiian Pizza	32273.25
Classic	The Pepperoni Pizza	30161.75
Supreme	The Spicy Italian Pizza	34831.25

----------********----------